Daily Scholar Papers Report — 2026-07-03

2026-07-03

Daily Scholar Papers Report — 2026-07-03

Download PDF

Window covered: 2026-07-02 → 2026-07-03 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Three Scholar-alert threads returned in the 24 h window: one “Recommended articles” batch (three candidates, all Stage-1 Skipped as saturated DL-for-vuln-detection at off-axis venues) plus two followed-researcher alerts (Emina Torlak, Armando Solar-Lezama). The followed-researcher override auto-proceeded both to Stage-2, but on retrieval both turned out to be historical works Scholar had freshly reindexed under the followed profile — Mariano et al.’s 2019 OOPSLA paper *Program Synthesis with Algebraic Library Specifications* and Porncharoenwase’s 2023 UW dissertation *Productivity Tools for Solver-Aided Programming*. Neither is a 2026 novelty, but both are canonical productivity-tools references in the solver-aided-programming lineage the reader tracks, and the deep-reads today are pitched as Keep-level reference summaries rather than Outstanding-level exegeses. The user-curated self-email queue was empty; the preference layer had no attributes clearing the n ≥ 2 threshold that matched either paper, so all classification was driven by the standard Stage-1 rubric plus the followed-researcher override.

Outstanding: 0 · **Keep:** 2 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	Program Synthesis with Algebraic Library Specifications	Mariano, Reese, Xu, Nguyen, Qiu, Foster, Solar-Lezama	PACMPL OOPSLA, Vol. 3, Art. 132, Oct 2019	DOI
1.2	Productivity Tools for Solver-Aided Programming	Porncharoenwase (advisor: Torlak)	UW PhD dissertation, 2023	PDF

1. Keep

1.1 · PROGRAM SYNTHESIS · PACMPL OOPSLA 2019 — algebraic rewrite-rule library specs are 53% the LoC of mocks and synthesise 2×–81× faster on 7/9 benchmarks; soundness and completeness proved for ordered non-unifiable rules

1.1 Program Synthesis with Algebraic Library Specifications — Mariano, Reese, Xu, Nguyen, Qiu, Foster, Solar-Lezama, *Proc. ACM Program. Lang.*, Vol. 3, No. OOPSLA, Article 132, October 2019

Authors. Benjamin Mariano, Josh Reese (both Maryland), Siyuan Xu (Purdue), ThanhVu Nguyen (Nebraska–Lincoln), Xiaokang Qiu (Purdue), Jeffrey S. Foster (Tufts), Armando Solar-Lezama (MIT). Solar-Lezama (MIT CSAIL) is the followed name; the byline gathers the North-American sketch-based-synthesis community around the Sketch/JSketch lineage.

Venue. PACMPL, Vol. 3, No. OOPSLA, Article 132 (October 2019), 25 pages, DOI 10.1145/3360558. Top venue. Scholar freshly reindexed this paper under Solar-Lezama's followed-researcher profile in the last 24 h — the deep-read below treats it as a canonical reference rather than a 2026-novel result.

Problem. Sketch-based synthesis works well when a synthesiser can reason about the program's environment, but every real-world program uses libraries, and library code is too big and too concrete for a synthesiser to inline. The standard workaround — hand-written *mock libraries* — sidesteps size but not concreteness: mocks reintroduce implementation details the synthesiser must reason about, and empirically the synthesiser struggles to prove properties across mock internals.

Approach — algebraic specifications instead of mocks. JLibSketch replaces mocks with *algebraic specifications*: rewrite rules over sequences of method calls. The paper's canonical example is a partial cryptography spec `decrypt(encrypt(m,k),k) ⇒ m` — encryption followed by decryption with the same key is the identity — which is dramatically smaller and more assertion-friendly than a mock crypto library. The implementation strategy is a compilation:

1. Extend JSketch's surface syntax with Java-like notation for algebraic specs.
2. Compile spec-using programs so that library calls are represented as **abstract data types (ADTs)** inside JSketch's existing object model.
3. Compile the rewrite rules to Sketch code that manipulates those ADTs. **No solver modifications required.**
4. Add explicit boxing/unboxing annotations to bridge JSketch's primitive/array/object distinction with the ADT representation.

The key design instinct is that the synthesiser only needs to reason about the *observable equational behaviour* of the library, not its implementation — and equational behaviour is exactly what a rewrite system encodes.

Formal result (paper's own statement). The paper formalises the pipeline as a compilation from `Sketch+libcore` (a core Sketch language extended with algebraic specifications) to `Sketchcore` (a core Sketch language without them). Its central theorem is quoted verbatim from Section 1: "We prove

that, if the rewriting rules are ordered and non-unifiable, then soundness and completeness hold, i.e., the Sketch+libcore program has a solution if and only if its compilation has a solution in Sketch." The ordered / non-unifiable side condition is what makes the term-rewriting system confluent enough for the compilation to preserve solutions.

Headline numbers.

- 9 benchmark programs synthesised across three library domains: data structures, cryptography, file systems.
- Algebraic specs are on **average 53%** of the lines of code of the corresponding mock library implementations.
- On **7 of 9** benchmarks, synthesis with algebraic specs is **2× to more than 81×** faster than with mocks. On the remaining 2 the two approaches perform equally.
- Artifact: `github.com/bmarwritescode/oopsla-19-artifact`.

Why Keep. The paper is genuinely first-rate — top venue, formal soundness/completeness with a clean side condition, credible empirical wins, and a permissive CC-BY licence. It is a Keep rather than an Outstanding on today's report because it is a 2019 paper freshly reindexed under the followed profile, not a 2026 result. Pair with today's second Keep (Porncharoenwase's dissertation) to see the Sketch → JSketch → JLibSketch → Rosette 4 / Leanette arc that the solver-aided productivity-tools neighbourhood has been running for a decade.

Reusability signals. The ADT-embedding trick — model library calls as terms of an algebraic data type, then let the solver reason over normal forms — is generic and should transfer to any synthesiser with algebraic-data-type support (Rosette, Sketch4J, more recent SyGuS-adjacent tools). The ordered / non-unifiable restriction is what limits the technique's reach; a project that could relax that restriction using more modern normalisation-by-evaluation or completion techniques from term rewriting would push the same design pattern into a wider slice of the library ecosystem.

Open questions. (i) Can the ordered / non-unifiable rule constraint be lifted with newer confluence techniques (Knuth–Bendix completion, higher-order rewriting)? (ii) How does the approach scale to library APIs with hundreds of methods and complex algebraic interactions (e.g., collections that combine ordering, uniqueness, and iteration semantics)? (iii) The 7/9 speedup range is wide — what makes the two tie-cases resist speedup, and is it a structural feature of the synthesis problem or an artefact of the benchmark?

1.2 · SOLVER-AIDED PROGRAMMING · UW dissertation 2023 — S_c symbolic-evaluation semantics with symbolic-factory interface; Rosette 4 validated against a Lean-verified Leanette via solver-aided differential testing; SymFix formalises symbolic performance repair

1.2 Productivity Tools for Solver-Aided Programming — Porncharoenwase, PhD dissertation, University of Washington, 2023

Author. Sorawee Porncharoenwase. Reading committee: Emina Torlak (chair), Xi Wang, Zachary Tatlock. Torlak (UW / now MIT) is the followed name. Scholar freshly reindexed the deposit under Torlak's profile in the last 24 h.

Venue. UW ResearchWorks doctoral dissertation, 2023. Not a conference paper — the individual research chapters have prior/companion papers at PLDI / OOPSLA. Treated on this report as a canonical unified retelling rather than a novel result.

License. UW ResearchWorks dissertation deposit — figures not embedded in this summary.

Thesis statement. *Recontextualisation* (making solver-aided programming feel familiar to programmers via language abstraction) and *extensibility* (letting programmers inject domain-specific knowledge to steer the tools) are the two design axes for productivity tools in this space. The dissertation supports the thesis with three contributions across the lifecycle of a solver-aided program.

Contribution 1 (Chapter 2) — A formal foundation for symbolic evaluation. Introduces λ_c, a core language for reusable symbolic evaluators, and S_c, a symbolic evaluation semantics with merging that is parameterised over a *symbolic factory interface*. Proves soundness and completeness of S_c and of the derived angelic-execution and verification queries. Two S_c-conforming implementations are built as a case study: **Leanette**, a Lean-verified reference implementation, and **Rosette 4**, a Racket-hosted optimised implementation. The dissertation then introduces *solver-aided differential testing* to validate Rosette 4 against Leanette — a solver looks for programs on which the two implementations diverge, giving the practitioner real confidence in an implementation that is not itself mechanised. Empirical section compares the Rosette 4 interface (Chapter 2.7.1) and performance (Chapter 2.7.2) against Rosette 3.

Contribution 2 (Chapter 3) — Scaling up: SymFix. Formalises *symbolic performance repair*: given a solver-aided program whose symbolic cost is too high, find a semantically equivalent variant with lower symbolic cost. Introduces the SymFix algorithm — a profile-guided search for fixes — and uses *symbolic reflection* to reason about the host language (Racket), letting the repair look inside macros and language constructs the naive approach cannot see.

Contribution 3 (Chapter 4) — An expressive pretty printer. Closes the human loop: solver-aided programs need readable output layouts to be usable by end users, and Chapter 4 develops a pretty printer that meets the expressiveness requirements the earlier chapters imply.

Why Keep. The dissertation aggregates already-published PLDI/OOPSLA work — the primary research disclosure happened at the conferences. Its value here is (a) the *unified thesis frame* (recontextualisation + extensibility) that no single conference paper carries, and (b) as a durable reference for anyone building a new symbolic evaluator or reasoning about the design space of solver-aided productivity tools. Historic reindex, so Keep rather than Outstanding.

Reusability signals. The `symbolic factory interface` is a candidate reference architecture for any project that wants to build a symbolic evaluator over an existing dynamic language — the point is that S_c is a *semantic contract* an implementation can conform to, not a monolithic implementation. A new backend (e.g., a symbolic evaluator for Rust MIR, or a JAX-native symbolic evaluator on top of Python) can plug into the S_c query family without redoing the correctness argument. SymFix's profile-guided formulation is worth having on the shelf in advance if any future project needs to treat symbolic-execution cost as a first-class optimisation target.

Open questions. (i) How much of the Leanette/Rosette 4 correspondence carries over to non-Racket hosts — is Lean-verified reference + differential testing a viable template for verifying a symbolic evaluator in a language whose semantics is less formally specified? (ii) SymFix's profile-guided search relies on a cost model; how sensitive is repair quality to the choice of cost model? (iii) The pretty-printer chapter's expressiveness requirements — do they generalise beyond solver-aided outputs to, say, LLM tool-call traces where an interactive layout also matters?

Cross-Paper Synthesis

Today's two Keep papers are both from the *solver-aided productivity tools* neighbourhood — Sketch/JSketch → Rosette/Leanette — and both surfaced as historical reindexes rather than fresh 2026 work. Read as a pair they trace a decade-long design instinct: **make the synthesiser's job smaller by giving it a cleaner semantic contract, then engineer the contract to be reusable across implementations.** JLibSketch (2019) does this at the library boundary — the algebraic specification is a semantic contract that the synthesiser can reason over cheaply, and the compilation shows the contract can be discharged mechanically. Porncharoenwase's S_c (2023) does the same trick at the evaluator boundary — the symbolic-factory interface is a semantic contract that any conforming evaluator (Leanette, Rosette 4, hypothetically others) can plug into without redoing the correctness argument. In both cases the win comes from separating the semantic contract from the implementation, then treating the implementation as replaceable. Anyone building a new synthesis or verification tool today should ask, on both boundaries, "what is the smallest, cleanest semantic contract my consumers actually need?" — and today's Skipped Recommended batch (three DL-classifier vulnerability-detection papers with no formal characterisation of what their classifiers are learning) is a useful negative example: those pipelines do not decompose along a semantic-contract boundary, and it shows in the papers' limited reusability.

Writing & Rationale Insights

The interesting rhetorical move in the Mariano et al. paper is the *ordered* / *non-unifiable* side condition on the theorem. A weaker paper would have proved a general soundness result and then noted in the discussion that some rules are hard to handle; JLibSketch instead pins the side condition on the theorem statement itself. This forces the reader to internalise the condition (rewrite rules must be ordered and non-unifiable) before internalising the guarantee, which sets up the empirical section: the benchmarks are chosen so the side condition holds. The paper is honest about what it can and cannot do, and gets the credit for its formal result without over-claiming. Porncharoenwase's dissertation makes the opposite rhetorical move at its scale: the *symbolic factory interface* is introduced as a semantic contract, then the two case studies (Leanette in Lean, Rosette 4 in Racket) prove by demonstration that the contract is implementable in radically different settings. The thesis narrative gains persuasive force from *two* implementations rather than one — a single Rosette 4 chapter would read as an engineering write-up, but the Rosette 4 + Leanette pair reads as evidence that the semantic contract is genuinely portable. Both papers illustrate the same lesson for our own writing: **state the load-bearing side condition on the theorem, and give the abstraction at least two concrete instantiations** — either move alone is worth less than the combination.